

TOOLS THAT HELP YOU HANDLE CONFIG FILES

Prepared By: Hosni Zaaraoui

Editing config files is one thing—but managing them smartly and safely is what really sets apart beginner admins from pros. Luckily, Linux offers powerful and lightweight tools to make your config journey easier.

Let's dive into the essential tools you should know

diff and colordiff — Compare Configuration Changes

Purpose:

See what changed between two versions of a file.

Command examples:



```
# Compare current vs backup SSH config to see changes
# Shows line-by-line differences between files in plain text
diff sshd_config sshd_config.bak

# Compare Nginx config versions with colored output
# Makes it easier to spot changes (additions/removals) at a glance
colordiff nginx.conf nginx.conf.old
```

How it helps:

- Before restarting a service, check your new config against the old.
- Debug unexpected behaviors by spotting wrong edits.

colordiff is simply a colored, more readable version of diff (install it with `sudo apt install colordiff`).

Tip: Always diff your configs before applying big changes, especially on production servers!

grep —

Find Settings Quickly in Large Files

Purpose:

Search for keywords inside config files.

Command examples:



```
grep PermitRootLogin /etc/ssh/sshd_config

#PermitRootLogin prohibit-password
# the setting of "PermitRootLogin without-password".
```



```
# Search for 'listen' directives in Nginx config (case-insensitive)
# Helps quickly find all ports/services Nginx is configured to listen on
grep -i listen /etc/nginx/nginx.conf
```

How it helps:

- Quickly find where a particular setting is.
- Case-insensitive search (-i) helps when you're not sure about capitalization.

Tip: Use `grep -r` to recursively search through a whole directory of configs!

crudini —

Edit .ini Files From the Command Line

Purpose:

Easily read, write, and modify INI files without manually opening them.

Command examples:



```
# Set the PHP memory_limit to 512MB in the PHP configuration file
# (Modifies the [PHP] section of /etc/php.ini)
crudini --set /etc/php.ini PHP memory_limit 512M

# Retrieve the current memory_limit value from the PHP configuration
# (Reads from the [PHP] section of /etc/php.ini)
crudini --get /etc/php.ini PHP memory_limit
```

How it helps:

- Scripts can modify config files on the fly.
- No risk of syntax errors or messy edits when automating changes.

Tip: Perfect for DevOps pipelines or when managing lots of INI configs across servers!

yq —

Work With YAML Files Like a Boss

Purpose:

Parse, edit, validate, and create YAML files.

Command examples:



```
# Enable DHCP for eth0 interface in Netplan config (modifies file directly  
with -i flag)
```

```
yq eval '.network.ethernets.eth0.dhcp4 = true' -i /etc/netplan/50-cloud-  
init.yaml
```

```
# List all service names from a Docker Compose file (useful to see what  
containers are defined)
```

```
yq eval '.services[].name' docker-compose.yml
```

How it helps:

- YAML files need precise indentation—yq keeps everything neat.
- Easily automate YAML changes without human error.

Tip: yq is for YAML what jq is for JSON—if you deal with Netplan, Kubernetes, or Ansible, yq is a must!

jsonlint and jq —

Validate and Manipulate JSON Files

Purpose:

Ensure your JSON files are valid and nicely formatted.

Command examples:



```
# Validate the JSON syntax of 'settings.json' to check for any formatting errors
jsonlint settings.json

# Pretty-print the contents of 'settings.json' for better human readability
jq '.' settings.json

# Extract just the DHCP settings from the 'network' section in 'config.json'
jq '.network.dhcp' config.json
```

How it helps:

- Catch missing commas, bad brackets, or misquoted keys.
- Format (pretty-print) or query JSON structures with ease.

Tip: Always validate your JSON after manual edits to avoid crashes or unexpected errors in applications.

